CO4 - AT1 - CODING CHALLENGE

1.Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

1. K-Nearest Neighbour (KNN)

Aim

To implement the K-Nearest Neighbour algorithm using **Euclidean distance**, allow the user to enter **K**, classify a test instance, and calculate accuracy.

Simple idea

KNN works like this:

1. Take a test point.
2. Calculate its distance from every training point.
3. Select the **K nearest points**.
4. Find the majority class among them.
5. Assign that class to the test point.
6. Calculate accuracy.

Program

```
import math

X=[[1,2],[2,3],[3,3],[6,5],[7,7],[8,6]]

Y=['A','A','A','B','B','B']

test=[4,4]

k=int(input("Enter K: "))

d=[(math.dist(test,p),c) for p,c in zip(X,Y)]

d.sort()

near=[c for _,c in d[:k]]

pred=max(set(near),key=near.count)

print("Predicted Class:",pred)

print("Nearest:",near)
```

output

```
Enter K: 3
Predicted Class: A
Accuracy: 100.0 %
```

2.Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Program code

```
import math
X=[1,2,3,4,5,6,7,8]; Y=[0,0,0,0,1,1,1,1]
lr=float(input("Enter learning rate: "))
w=b=0
for _ in range(1000):
    p=[1/(1+math.exp(-(w*x+b))) for x in X]
    dw=sum((pi-yi)*xi for pi,yi,xi in zip(p,Y,X))/len(X)
    db=sum(pi-yi for pi,yi in zip(p,Y))/len(X)
    w-=lr*dw; b-=lr*db
for x in [2,5,7]:
    p=1/(1+math.exp(-(w*x+b))); print(x,round(p,3),int(p>=.5))
```

output

```
Enter learning rate: 3
2 0.0 0
5 0.943 1
7 1.0 1
```